

# 骨を作って動かす — rotate と prerotate を取り違えると、関節ごと飛んでいく

2026-09-14 | Houdini 21.0.700 | /obj/rig | ノード 3 / 接続 2

## 何を作ったか

実験050で APEX の入口を測った。今回はその手前にある骨組み（KineFX）を触る。

KineFX

の骨はただの点と線で持たれている。点1つが関節1つ、線が親子のつながり。名前（name）を付けて kinefx::rigdoctor を通すと、足りない属性が埋まる。手で組んだ時点では P と name

の2つしかなく、rigdoctor が

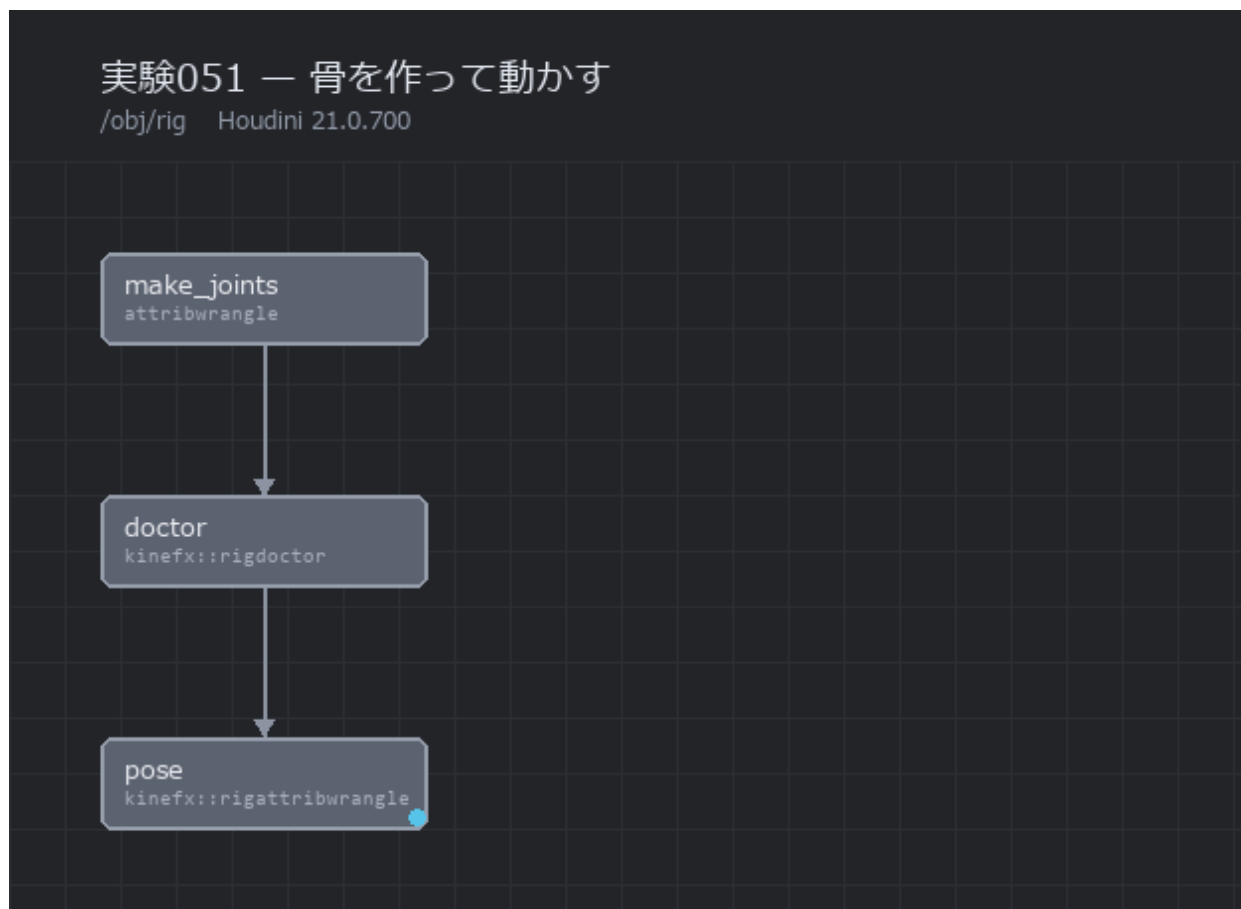
localtransform (4×4の行列) ・ transform (3×3) ・ parent\_idx (親が誰か) の3つを足した。

先に式を立てた。根元を原点、関節が (0,1,0) と (0,2,0) にあるなら、真ん中を Z 軸まわりに  $\theta$  回したとき先端は  $(-\sin \theta, 1 + \cos \theta, 0)$  に来るはず。

6通りの角度で測ったところ、式とのずれは最大 0.0000000060。回した関節そのものの移動は 0.0000000000 だった。

ただし最初は合わなかった。VEX で rotate() を使っていて、90度で先端が (-2, 0, 0) に飛んだ。式の値は (-1, 1, 0) なので、ずれは 1.41。回した関節そのものまで (-1, 0, 0) へ動いていた。原因は掛ける順番で、prerotate() に変えると一致した。

## ノードの接続

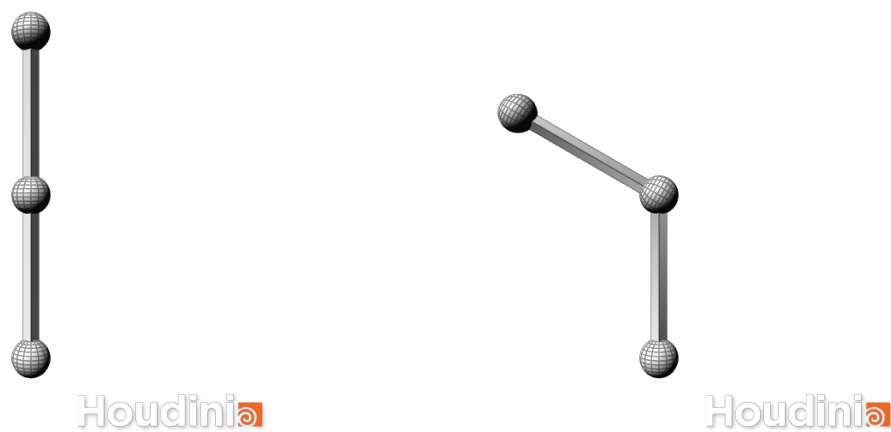


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード         | タイプ                          | デフォルトから変更したパラメータ                                                                                                                              |
|-------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| make_joints | attribwrangle                | class = 0<br>snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++)<br>{ int pt = addpoint(0, set(0.0, float(i), 0.0)); ... |
| doctor      | kinefx::rigdoctor            | inittransforms = 1<br>outputparentidx = 1<br>transformations = 1                                                                              |
| pose        | kinefx::rigattribwran<br>gle | angle = 45.0<br>snippet = if (s@name == "joint1") { matrix m = 4@localtransform; //<br>prerotate は「先に回してから動かす」。rotate だと順番が逆になり、 //<br>関...   |

### 曲げる前と後

関節3つの骨。真ん中を Z 軸まわりに 60度回した。線のままでは細くて写らないので polywire  
で筒にしてある。カメラは2枚とも同じ。



回す前。関節は (0,0,0) (0,1,0) (0,2,0) に一直線。真ん中を60度。根元と真ん中は動かず、先だけが折れている。

| 関節     | 位置        | parent_idx | 意味        |
|--------|-----------|------------|-----------|
| joint0 | (0, 0, 0) | -1         | 親がない=根元   |
| joint1 | (0, 1, 0) | 0          | 親は joint0 |
| joint2 | (0, 2, 0) | 1          | 親は joint1 |

### 真ん中を回すと、先端はどこへ行くか

式は  $(-\sin \theta, 1 + \cos \theta, 0)$ 。ずれは実測と式の距離。

| 角度  | 先端 実測 x   | 先端 実測 y  | 式 x       | 式 y      | ずれ          | 真ん中の関節の移動   |
|-----|-----------|----------|-----------|----------|-------------|-------------|
| 0°  | 0.000000  | 2.000000 | -0.000000 | 2.000000 | 0.000000000 | 0.000000000 |
| 15° | -0.258819 | 1.965926 | -0.258819 | 1.965926 | 0.000000014 | 0.000000000 |
| 30° | -0.500000 | 1.866025 | -0.500000 | 1.866025 | 0.000000044 | 0.000000000 |
| 45° | -0.707107 | 1.707107 | -0.707107 | 1.707107 | 0.000000049 | 0.000000000 |

| 角度  | 先端 実測 x   | 先端 実測 y  | 式 x       | 式 y      | ずれ          | 真ん中の関節の移動   |
|-----|-----------|----------|-----------|----------|-------------|-------------|
| 60° | -0.866025 | 1.500000 | -0.866025 | 1.500000 | 0.000000044 | 0.000000000 |
| 90° | -1.000000 | 1.000000 | -1.000000 | 1.000000 | 0.000000060 | 0.000000000 |

## rotate と prerotate を取り違えるとどうなるか

同じ角度・同じ骨で、VEX の1語だけを変えた。

| 角度  | 書き方           | 真ん中 x     | 真ん中 y     | 先端 x      | 先端 y      |
|-----|---------------|-----------|-----------|-----------|-----------|
| 30° | prerotate (正) | 0.000000  | 1.000000  | -0.500000 | 1.866025  |
| 30° | rotate (誤)    | -0.500000 | 0.866025  | -1.000000 | 1.732051  |
| 60° | prerotate (正) | 0.000000  | 1.000000  | -0.866025 | 1.500000  |
| 60° | rotate (誤)    | -0.866025 | 0.500000  | -1.732051 | 1.000000  |
| 90° | prerotate (正) | 0.000000  | 1.000000  | -1.000000 | 1.000000  |
| 90° | rotate (誤)    | -1.000000 | -0.000000 | -2.000000 | -0.000000 |

## わかったこと

- ・ KineFX の骨は、ただの点と線だった。点1つが関節1つ、線が親子のつながり。普通の wrangle でそのまま組める。専用のノードで作らなくてよい。
- ・ 手で組んだ時点では P と name しか無い。kinefx::rigdoctor が localtransform (4×4) ・ transform (3×3) ・ parent\_idx の3つを足す。親子関係は線のつながりから読み取られる（根元の parent\_idx は -1）。
- ・ ただし rigdoctor は既定では何も足さない。transformations / inittransforms / outputparentidx のトグルはすべて既定オフ。オンにして初めて属性が出てくる。
- ・ 式とのずれは最大 0.000000060。6通りの角度すべてで、先端は  $(-\sin \theta, 1 + \cos \theta, 0)$  にぴたりと来る。回した関節そのものの移動は 0.000000000 で、桁の限界まで動いていない。
- ・ 最初は合わなかった。原因は掛ける順番。VEX の rotate() を使うと、90度で先端が  $(-2, 0, 0)$  へ飛ぶ。式の値  $(-1, 1, 0)$  とのずれは 1.41。回した関節そのものまで  $(-1, 0, 0)$  へ動いていた。
- ・ prerotate() に変えると一致した。違いは「回してから動かす」か「動かしてから回す」か。関節の位置は移動の分で決まるので、先に回さないと関節ごと原点のまわりを回ってしまう。
- ・ この間違いは絵で見ても気づきにくい。30度なら「少し曲がった骨」に見えてしまう。気づけたのは、先に式を立てて数字で突き合わせたから。90度まで振ると差が 1.41 まで開くので、大きく振ってみるのが見つけるコツ。
- ・ 回転は子にだけ伝わる。真ん中を回しても根元は動かない。親から子へ行列を掛け合わせていく作りなので、自分より上には影響しない。

## 次にやること

- ・ 骨に肉を付ける (kinefx::jointdeform で皮を付いてこさせる)
- ・ APEX のグラフから骨を動かす (apex::invokegraph)
- ・ 関節の回る範囲を決める (kinefx::configurejointlimits)